IMA

Inventory Management Application

USER MANUAL · v1.0

Database: inventory_system · Backend: Java · UI: JavaFX · DB: MySQL 8.0

Roles: Manager · HOD · Dean

Table of Contents

01 Introduction

What IMA is, what it solves, and who uses it

1.1 Overview

The **Inventory Management Application (IMA)** is a desktop software system built on **Java + JavaFX + MySQL** that digitizes and automates inventory operations. It connects to the **inventory_system** database at `jdbc:mysql://127.0.0.1:3306/inventory_system` and replaces manual registers and spreadsheets with a structured, real-time platform.

Every transaction — procurement via orders and bills, issuance to departments, returns from users, and returns to suppliers — is instantly reflected in the database, ensuring all users see accurate, live data at all times.

■ Real-Time Stock

qty_in_stock updated on every Add, Issue, and Return.

■ 3-Role RBAC

Manager · HOD · Dean — each with distinct permissions.

■ Full Audit Trail

Every issue, order, bill, and return is permanently logged.

1.2 Purpose

IMA is built to

- ■ Replace manual stock registers and error-prone spreadsheets
- ■ Automate stock updates — add, issue, return all reflected instantly
- ■ Enforce role-based access: Managers handle operations; HOD/Dean handle oversight
- ■ Maintain full transaction history for auditing and reporting
- ■ Track budget and financial usage through bill-linked procurement
- ■ Provide weekly and summary reports for informed decision-making

1.3 Scope

IMA covers the complete inventory lifecycle within an organization — from order placement and bill processing through issuance, return, and financial reporting. It is deployable in educational institutions, corporate offices, and small-scale industries.

1.4 Roles & Users

IMA uses a 3-role system defined in the `role` table and assigned via the `user` table:

User ID	User Name	Role	Access
M01	Himanshu Sir	Manager (role_id=1)	Update · Issue · Return · View modules
H01	Rajiv Sir	HOD (role_id=2)	Reports · Budget Analysis · Approvals
D01	CK Jha Sir	Dean (role_id=3)	Reports · Budget Analysis · Approvals

■ *The Manager's accessible product types are defined in the manager_product_type table. M01 is currently assigned STN and Misc categories.*

02 System Overview

Technology stack, three-layer architecture, workflow, and requirements

2.1 Technology Stack

Layer	Technology	Key Role in IMA
Backend Logic	Java (JDK)	Core processing, DB queries, validation, session management
User Interface	JavaFX	All screens: Login, Dashboard, modules, forms, tables
Database	MySQL 8.0 — inventory_system	Stores all records; enforces FK constraints and CHECK rules
DB Connection	DBConnection.java	Singleton connector: jdbc:mysql://127.0.0.1:3306/inventory_system

2.2 Three-Layer Architecture

PRESENTATION LAYER	**JavaFX GUI** LoginUI · DashboardUI · ManageInventoryUI · IssueUI · ReturnUI · ReportUI · BudgetAnalysisUI
APPLICATION LAYER	**Java (Business Logic)** Validation · RBAC · Stock calculation · Session management · DB operations
DATA LAYER	**MySQL — inventory_system** role · user · product · product_type · supplier · order_table · bill_invoice · issue · return_table · rts_table

■ *DBConnection.java provides a shared MySQL connection. All UI classes call DBConnection.getConnection() — no direct JDBC handling in UI layers.*

2.3 Application Workflow

1 Launch	2 Login	3 Auth	4 Dashboard	5 Module	6 Operate	7 DB Update	8 Output
App starts	Enter credentials	DB verifies	Role-based view	Select task	Perform action	Records saved	Screen/report

2.4 System Requirements

Hardware

- Minimum 4 GB RAM
- 2 GHz Processor
- Standard keyboard, mouse, monitor

Software

- OS: Windows (recommended)
- Java JDK (configured in PATH)
- MySQL 8.0 server running
- JavaFX runtime libraries
- inventory_system DB created and seeded

03 Login & Authentication

Secure entry point using the user table in inventory_system

3.1 Purpose

The Login module is IMA's first security layer. It queries the `user` table (joined with `role`) to verify credentials and determine which modules the user may access. Passwords are stored as plain text in the current implementation; fields are trimmed before comparison.

3.2 Credentials & Roles

user_id	user_name	password	role	Accessible Modules
M01	Himanshu Sir	pass123	Manager	Update, Issue, Return (User), View
H01	Rajiv Sir	pass123	HOD	Reports, Budget Analysis, Approvals
D01	CK Jha Sir	pass123	Dean	Reports, Budget Analysis, Approvals

■ ***Note:*** *These are default credentials. Passwords are case-sensitive and trimmed before comparison (SET SQL_SAFE_UPDATES handled in DB setup). Change after first login.*

3.3 Authentication Flow

1 Enter	2 Trim	3 Query	4 Join	5 Grant/Deny	6 Dashboard
user_id + password	Strip whitespace	SELECT from user	Match role table	Access or error	Role-filtered view

3.4 Validation Rules

Login Validation

- ■ Both user_id and password fields are mandatory — empty fields blocked
- ■ user_id must exist in the user table
- ■ Password comparison is exact and case-sensitive

■ Failed login shows a descriptive error alert

■ No lockout mechanism in v1.0 — planned for future release

3.5 Password Recovery

Each user has a `secret_question` and `secret_answer` stored in the user table. All three users are currently set to 'What is your senior secondary school name?'

1 Click	2 Display	3 Answer	4 Verify	5 Reset
Retrieve Password	Security question shown	User enters answer	DB match check	Password update allowed

■ *secret_answer values: M01=xyz, D01=abc, H01=def (from DB setup)*

04 Dashboard Module

Central hub — role-filtered navigation after successful login

4.1 Overview

DashboardUI.java is the first screen shown after successful login. It displays only the modules the logged-in user's role can access. The role is determined from the role_id in the user table and passed through the session.

4.2 Module Access by Role

Module	Manager (M01)	HOD (H01)	Dean (D01)
Update (Manage Inventory)	■	■	■
Issue Items	■	■	■
Return from User	■	■	■
Return to Supplier	■	■	■
View / Manage Inventory	■	■	■
Report	■	■	■
Budget Analysis	■	■	■
Request Inventory	■	■	■

4.3 Logout

The Logout button terminates the active session and returns to LoginUI. No session token is persisted — each login is independent.

05 System Modules

Detailed guide to every functional module — UI class → DB table mapped

5.1 Update Module (ManageInventoryUI.java)

The Update Module is the inventory control center. It is implemented as `ManageInventoryUI.java` and organized into **four tabs**: Products, Suppliers, Orders, and Bills. From here, the Manager can add new items, log supplier bills, modify records, place orders, and perform soft-deletes.

Add Inventory — `AddIventoryUI.java`

- Adds a new product to the product table with a VARCHAR(6) pid (e.g., P0001)
- Fields: product_name, description, qty_in_stock, min_qty_required (≥5), ptype_id
- Manager can only add products in their assigned ptype_id (from manager_product_type)
- Stock count increases immediately; qty_updated_date removed in v1.0 schema

Bill Received — `Bill.java + bill_invoice table`

- Records a bill linked to an existing order via entry_id (FK to order_table)
- Fields: bill_no (UNIQUE), bill_received_by, bill_amount, qty_received, received_date
- Automatically increases qty_in_stock in the product table
- bill_status set to COMPLETE when total qty_received ≥ qty_ordered; else INCOMPLETE
- order_status updated to PAID when bill is complete; else stays IN_PROCESS
- Soft-delete reverses stock and recalculates statuses via recalculateStatuses()

Modify — `ModifyProductUI.java`

- Edit product_name, description, min_qty_required directly in the Products tab table
- qty_in_stock and ptype_id are read-only — not editable from this module
- Edit blocked if active bills exist for that entry (data integrity protection)
- Supplier fields editable: name, email, contact_no, address (ptype_id read-only)

Order Placed — `AddOrderUI.java → order_table`

- Creates a new row in order_table: order_no, pid, sid, order_date, qty_ordered
- Default order_status = IN_PROCESS; record_status = ACTIVE

- Cannot delete an order while active (record_status=ACTIVE) bills are linked
- Soft-delete sets record_status = INACTIVE (data never physically removed)

Delete

`DeleteInventoryUI.java`

- Soft-delete only — sets record_status = INACTIVE in order_table or bill_invoice
- Deleting a bill: first soft-deletes, then reverses stock (qty_in_stock -= qty_received)
- Then recalculates order_status and bill_status for remaining ACTIVE bills
- Orders with active bills cannot be deleted — system blocks with error message

5.2 Issue Module (IssueUI.java → issue table)

Issues inventory items to users or departments. Each issuance creates a row in the `issue` table and decrements `product.qty_in_stock`. The `qty_returned` column (added later) tracks how much of an issued batch has been returned.

DB Column	Type	Description	Required
pid	VARCHAR(6)	Product being issued (FK → product)	Yes
issue_to	VARCHAR(50)	Name of recipient	Yes
issued_by	VARCHAR(50)	Person processing the issue	Yes
dept_name	VARCHAR(50)	Receiving department	Yes
qty_issued	INT > 0	Quantity to issue	Yes
reason	VARCHAR(100)	Purpose of issuance	Optional
date	DATE	Issue date	Yes
qty_returned	INT DEFAULT 0	Cumulative quantity returned so far	Auto

1 Select	2 Enter	3 Validate	4 Insert	5 Update
Choose product	Fill all fields	Check stock	issue table row	Stock decreases

■ ***Note:*** *qty_in_stock is checked before issuing. If qty_issued > available stock, the operation is blocked. qty_issued must be > 0 (DB CHECK constraint enforced).*

5.3 Return Module — Return from User (ReturnUI.java → return_table)

Handles items returned by staff or departments. References an original `issue_id` and increases `qty_in_stock`. The `issue.qty_returned` column is also updated.

DB Column	Type	Description
return_id	INT AUTO_INCREMENT	Primary key
ptype_id	VARCHAR(5)	Product type (FK → product_type)

DB Column	Type	Description
issue_id	INT	Reference to original issue (FK → issue)
pid	VARCHAR(6)	Product being returned (FK → product)
quantity	INT > 0	Quantity returned
return_to	VARCHAR(50)	Where item is returned (e.g., Store)
date	DATE	Return date

■ *quantity returned cannot exceed qty_issued - qty_already_returned for that issue_id. Stock increases by the returned amount.*

5.4 Return to Supplier (rts_table)

Handles defective or excess items returned to a supplier. Linked to a `bill_id` for financial traceability. Stock **decreases** when items are returned to supplier.

DB Column	Type	Description
rts_id	INT AUTO_INCREMENT	Primary key
bill_id	INT	FK → bill_invoice (which bill this return relates to)
pid	VARCHAR(6)	Product being returned (FK → product)
sid	INT	Supplier receiving the return (FK → supplier)
quantity	INT > 0	Quantity sent back
rts_date	DATE	Date of return
record_status	ENUM(SENT, RECEIVED)	Delivery status — default SENT

■ ***Note:*** *This is a separate table (rts_table) from return_table. Return from User → return_table (stock increases). Return to Supplier → rts_table (stock decreases).*

5.5 View / Manage Inventory (ManageInventoryUI.java — 4 Tabs)

Organized into four tabs, this module provides read and limited-edit access to core records. All tabs use live DB queries — no caching.

Products Tab	`product`	View/edit product_name, description, min_qty_required. qty_in_stock & ptype_id are read-only.

Suppliers Tab	`supplier`	View/edit name, email, contact_no, address. ptype_id is read-only.
Orders Tab	`order_table`	View active orders (record_status=ACTIVE). Delete only if no active bills linked.
Bills Tab	`bill_invoice`	View active bills with amount, qty_received, status. Delete reverses stock automatically.

5.6 Report Module (ReportUI.java → ReportData.java)

Accessible to HOD and Dean. Generates reports by querying issue, return_table, bill_invoice, and order_table.

Report Types

- ■ Weekly Report — inventory movement for a selected week (issues, returns, bills)
- ■ Summary Report — overall snapshot of stock, procurement, and issued items

5.7 Budget Analysis Module (BudgetAnalysisUI.java → BudgetService.java)

Tracks financial usage by summing bill_amount from `bill_invoice` where record_status = ACTIVE. Accessible to HOD and Dean only.

Component	Source	Description
Total Budget	Configured / entered	Total funds allocated for procurement
Amount Spent	SUM(bill_amount) from bill_invoice	All active bill amounts totalled
Remaining Budget	Total - Spent	Live calculated remaining balance

5.8 Request Inventory Module

Allows users to request items not currently in stock. Requests are reviewed and approved by HOD/Dean before an order is placed.

1 Submit	2 Review	3 Approve	4 Order
User files request	HOD/Dean reviews	Request approved	Order placed in order_table

06 Database Design

All 10 tables in inventory_system — columns, types, purpose, and relationships

The `inventory_system` database contains 10 tables. Stock is tracked directly in `product.qty_in_stock` — there is no separate 'Current Table'. All deletes are soft (record_status = INACTIVE). product IDs are `VARCHAR(6)` (e.g., P0001).

role

Defines the three user roles

Column	Type	Description
`role_id`	`INT PK`	Role identifier: 1=Manager, 2=HOD, 3=Dean
`role_name`	`VARCHAR(20) UNIQUE`	Role label stored in DB

user

All system users with credentials and role assignment

Column	Type	Description
`user_id`	`VARCHAR(5) PK`	Login identifier e.g. M01, H01, D01
`user_name`	`VARCHAR(30)`	Display name of the user
`password`	`VARCHAR(30)`	Login password (plain text, trimmed before compare)
`role_id`	`INT FK→role`	Determines access level
`secret_question`	`VARCHAR(255)`	Used for password recovery
`secret_answer`	`VARCHAR(255)`	Answer verified during recovery
`email`	`VARCHAR(100) UNIQUE`	User email (added later — optional)

product_type

Lookup table for product categories

Column	Type	Description
`ptype_id`	`VARCHAR(5) PK`	Category code: STN, FUR, CS, Misc
`ptype_name`	`VARCHAR(30) UNIQUE`	Full category name e.g. Stationery

manager_product_type

Maps which product types a Manager can manage

Column	Type	Description
user_id	VARCHAR(5) PK,FK→user	Manager's user_id
ptype_id	VARCHAR(5) PK ,FK→product_ type	Allowed product type
—	(composite PK)	M01 currently manages STN and Misc

supplier

Supplier directory — one supplier per product type

Column	Type	Description
sid	INT AUTO_INCR EMENT PK	Supplier ID
name	VARCHAR(50)	Supplier name
email	VARCHAR(50) UNIQUE	Contact email
contact_no	VARCHAR(15)	Phone number
address	VARCHAR(100)	Supplier address
ptype_id	VARCHAR(5) FK →product_typ e	What category this supplier handles

product

Core inventory table — every item lives here

Column	Type	Description
pid	VARCHAR(6) PK	Product ID e.g. P0001, P0002
product_name	VARCHAR(50)	Item name
description	VARCHAR(100)	Item description
qty_in_stock	INT ≥ 0	Current available stock — updated on every transaction
min_qty_required	INT ≥ 5	Low-stock threshold (DB CHECK constraint)
ptype_id	VARCHAR(5) FK →product_typ e	Product category

supplies

Junction table linking suppliers to products they supply

Column	Type	Description
sid	INT PK,FK→supplier	Supplier reference
pid	VARCHAR(6) PK,FK→product	Product reference
—	(composite PK)	Many-to-many: one supplier can supply many products

order_table

Tracks purchase orders placed with suppliers

Column	Type	Description
entry_id	INT AUTO_INCREMENT PK	Order entry identifier
order_no	VARCHAR(20)	Human-readable order number e.g. ORD201
pid	VARCHAR(6) FK→product	Ordered product
sid	INT FK→supplier	Supplier the order is placed with
order_date	DATE	Date order was placed
qty_ordered	INT > 0	Quantity ordered
order_status	ENUM(PAID,IN_PROCESS)	PAID when all qty received; else IN_PROCESS
record_status	ENUM(ACTIVE,INACTIVE)	INACTIVE = soft-deleted

bill_invoice

Bills received against orders — drives stock increases

Column	Type	Description
bill_id	INT AUTO_INCREMENT PK	Bill identifier
bill_no	VARCHAR(20) UNIQUE	Bill number e.g. B5001
bill_received_by	VARCHAR(50)	Who received the bill
bill_amount	DECIMAL(10,2) > 0	Bill value in currency
pid	VARCHAR(6) FK→product	Product in this bill
sid	INT FK→supplier	Supplier issuing the bill
entry_id	INT FK→order_table	Linked order
qty_received	INT > 0	Quantity received in this delivery
received_date	DATE	Date bill/goods were received
bill_status	ENUM(COMPLETE,INCOMPLETE)	COMPLETE when order fully fulfilled
record_status	ENUM(ACTIVE,INACTIVE)	INACTIVE = soft-deleted

issue — *Records every item issued to a department or user*

Column	Type	Description
issue_id	INT AUTO_INCREMENT PK	Issue identifier
pid	VARCHAR(6) FK→product	Issued product
issue_to	VARCHAR(50)	Recipient name
issued_by	VARCHAR(50)	Who processed the issue
dept_name	VARCHAR(50)	Receiving department
qty_issued	INT > 0	Quantity issued
reason	VARCHAR(100)	Purpose / reason (optional)
date	DATE	Issue date
qty_returned	INT DEFAULT 0	Running total of returned qty for this issue

return_table — *Return from user — items coming back from departments*

Column	Type	Description
return_id	INT AUTO_INCREMENT PK	Return identifier
ptype_id	VARCHAR(5) FK→product_type	Product category
issue_id	INT FK→issue	Original issue being reversed
pid	VARCHAR(6) FK→product	Returned product
quantity	INT > 0	Quantity returned
return_to	VARCHAR(50)	Destination e.g. Store
date	DATE	Return date

rts_table *Return to supplier — defective/excess items sent back*

Column	Type	Description
rts_id	INT AUTO_INCREMENT PK	RTS identifier
bill_id	INT FK→bill_invoice	Related bill for this return
pid	VARCHAR(6) FK→product	Product returned
sid	INT FK→supplier	Supplier receiving the return
quantity	INT > 0	Quantity sent back
rts_date	DATE	Date sent
record_status	ENUM(SENT,RECEIVED)	Delivery tracking — default SENT

Quantity Flow Logic

Every operation has a predictable, consistent effect on `product.qty_in_stock`:

Action	Table	Effect on qty_in_stock	Status Change
Add Inventory	product (INSERT)	■ INCREASES	—
Bill Received	bill_invoice (INSERT)	■ INCREASES by qty_received	order_status → PAID if complete

Action	Table	Effect on qty_in_stock	Status Change
Issue Items	issue (INSERT)	■ DECREASES by qty_issued	—
Return from User	return_table (INSERT)	■ INCREASES by quantity	issue.qty_returned updated
Return to Supplier	rts_table (INSERT)	■ DECREASES by quantity	record_status: SENT→RECEIVED
Delete Bill (soft)	bill_invoice (UPDATE)	■ REVERSES — qty decreases	order_status recalculated

■ ***Note:*** *Stock is tracked in product.qty_in_stock only — there is no separate Current Table. qty_in_stock has a CHECK constraint: qty_in_stock >= 0. min_qty_required has CHECK: min_qty_required >= 5.*

07 Input Validation

Data integrity rules enforced at both application and DB level

Validation Type	Rule	Example	Enforced By
Mandatory Fields	All required fields must be filled	pid, product_name cannot be empty	Java (pre-submit)
Data Type	Value must match column type	qty_issued must be integer	Java + MySQL
Range / CHECK	Value within DB constraint	qty_in_stock ≥ 0; min_qty_required ≥ 5	MySQL CHECK
Uniqueness	No duplicate keys	bill_no must be UNIQUE	MySQL UNIQUE KEY
FK Constraint	Referenced record must exist	pid must exist in product table	MySQL FK
Business Rule	Custom logic in Java	Can't delete order with active bills	Java (ManageInventoryUI)
Stock Check	qty_issued ≤ qty_in_stock	Cannot issue more than available	Java (IssueUI)

DB-Level Constraints (from db.sql)

08 Error Handling

Graceful recovery — all errors shown via JavaFX Alert dialogs

Error Type	Cause	Example	Response
Input Error	Missing/wrong data	Empty product name field	Alert shown; submit blocked
Auth Error	Invalid credentials	Wrong password for M01	Error message; login blocked
Constraint Error	Business rule violation	Delete order with active bills	Alert: 'Delete all bills first'
Stock Error	Issue > available stock	Issue 300 when only 200 in stock	Alert; operation cancelled
FK Error	Referenced ID missing	pid not in product table	SQLException caught; Alert shown
System Error	DB connection failure	MySQL server not running	SQLException: connection refused

Error Handling Principles

- All errors caught in try-catch blocks and shown via JavaFX Alert.AlertType.ERROR
- Invalid operations blocked before DB write — no partial commits
- Soft-delete errors (e.g., active bill blocks order delete) shown with specific guidance
- DB constraint violations caught as SQLExceptions and surfaced to the user
- System never crashes — all paths have error handlers

09 Security Features

RBAC, session control, password recovery, and data protection

Role-Based Access Control (RBAC)	Access is granted based on role_id from the user table. Manager (role_id=1) accesses operational modules. HOD (role_id=2) and Dean (role_id=3) access reporting and approval functions. The manager_product_type table further restricts which product categories a Manager can manage.
Credential Authentication	LoginUI queries the user table using the entered user_id. Passwords are trimmed and compared exactly. No login is possible without a matching user_id + password pair in the DB.
Session Management	Sessions are active only after successful login. Logout terminates the session and redirects to LoginUI. No persistent session tokens — each launch requires re-authentication.
Soft-Delete Data Protection	No records are physically deleted from the DB. Deletes set record_status = INACTIVE. This preserves audit history and prevents accidental data loss.
Password Recovery	secret_question and secret_answer in the user table enable recovery. The answer is verified against the stored value before allowing a reset. Recovery is impossible without the correct answer — no email-based reset in v1.0.

10 Conclusion

Summary, key benefits, and the value IMA delivers

10.1 Summary

IMA is a complete, role-driven Java desktop application that manages the full inventory lifecycle — procurement (orders + bills), issuance to departments, user and supplier returns, financial tracking, and reporting — through a single, consistent interface backed by a well-structured MySQL database.

10.2 Key Benefits

✓ **Accuracy**
DB CHECK constraints and Java-level validation together prevent bad data from entering the system.

✓ **Real-Time Stock**
product.qty_in_stock is updated immediately on every add, issue, bill, and return — no lag.

✓ **Accountability**
Full audit trail via issue, bill_invoice, return_table, and rts_table — nothing is physically deleted.

✓ **Role Enforcement**
3-role RBAC (Manager/HOD/Dean) plus product-type-level restrictions for Managers.

✓ **Scalability**
Three-layer architecture (JavaFX / Java / MySQL) allows independent scaling of each layer.

✓ **Financial Visibility**
Budget Analysis sums active bill_amounts in real time; HOD/Dean always know spend vs. budget.

IMA — inventory_system · Java · JavaFX · MySQL 8.0
A reliable, accurate, and transparent inventory management system built for real organizational use.